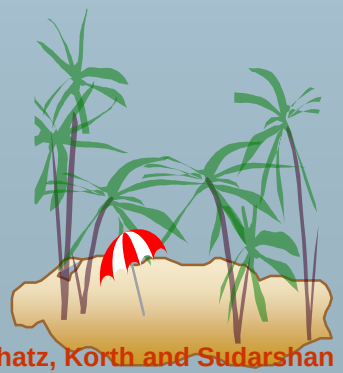


Chapter 12: Indexing (B+-trees)

- Basic Concepts
- Ordered Indices
- B+-Tree Index Files
- B-Tree Index Files
- Static Hashing
- Dynamic Hashing
- Comparison of Ordered Indexing and Hashing
- Index Definition in SQL
- Multiple-Key Access



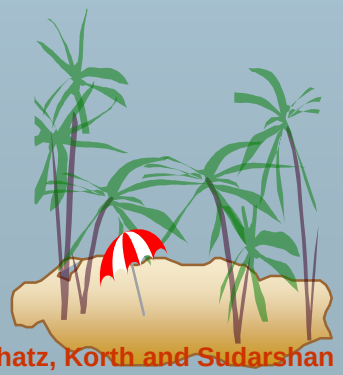
©Silberschatz, Korth and Sudarshan

Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
  - E.g., author catalog in library
- **Search Key** - attribute used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer

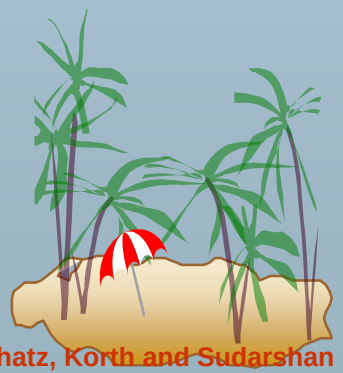
- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  - **Ordered indices:** search keys are stored in sorted order
  - **Hash indices:** search keys are distributed uniformly across "buckets" using a "hash function".



©Silberschatz, Korth and Sudarshan

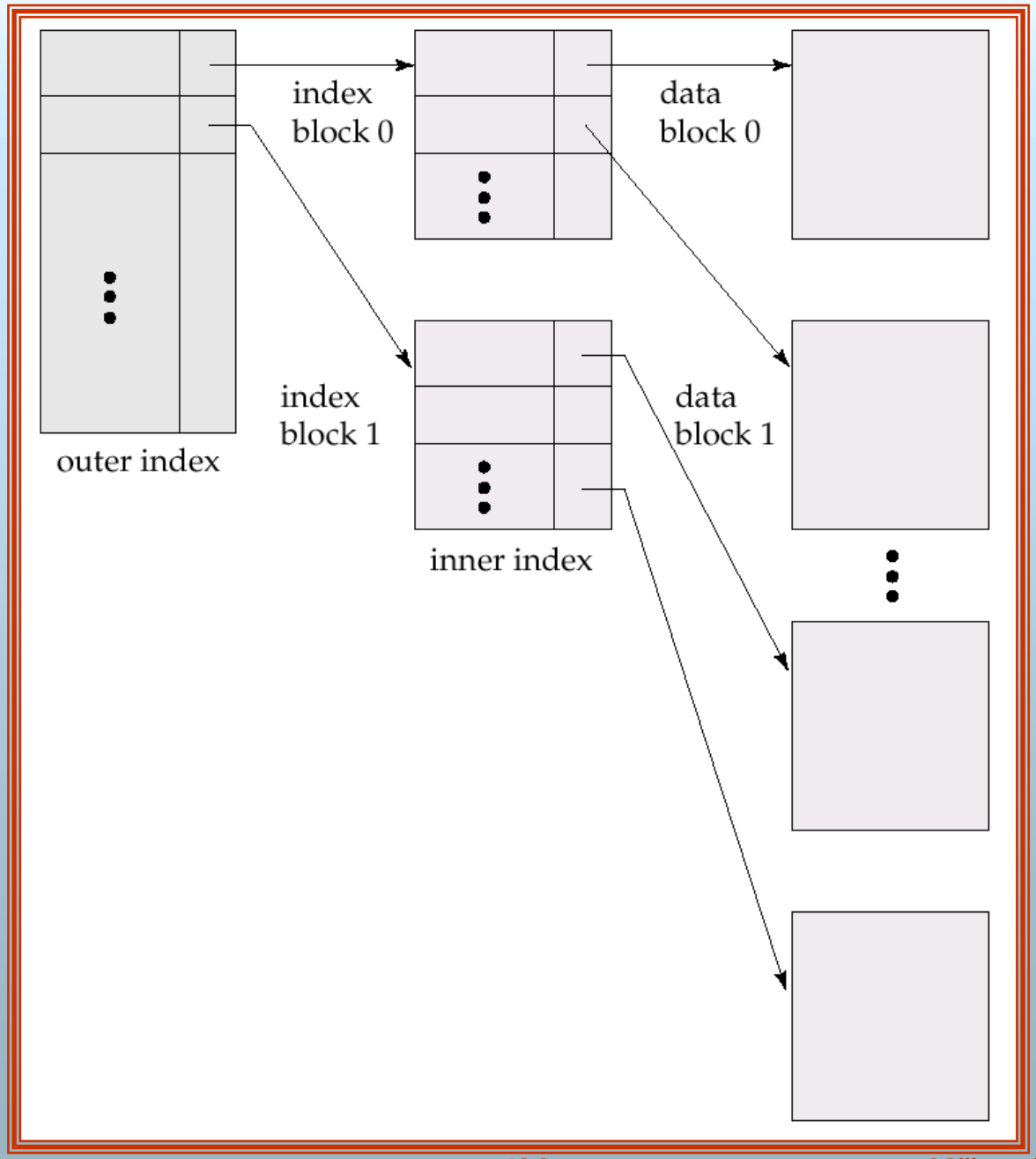
Multilevel Index

- If primary index does not fit in memory, access becomes expensive.
- To reduce number of disk accesses to index records, treat primary index kept on disk as a sequential file and construct a sparse index on it.
  - outer index – a sparse index of primary index
  - inner index – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.



©Silberschatz, Korth and Sudarshan

Multilevel Index (Cont.)

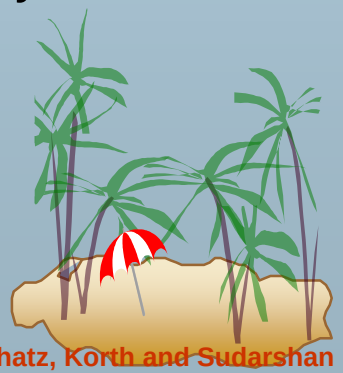


©Silberschatz, Korth and Sudarshan

B^+-Tree Index Files

B^+-tree indices are an alternative to indexed-sequential files.

- Disadvantage of indexed-sequential files: performance degrades as file grows, since many overflow blocks get created. Periodic reorganization of entire file is required.
- Advantage of B^+-tree index files: automatically reorganizes itself with small, local, changes, in the face of insertions and deletions. Reorganization of entire file is not required to maintain performance.
- Disadvantage of B^+-trees: extra insertion and deletion overhead, space overhead.
- Advantages of B^+-trees outweigh disadvantages, and they are used extensively.

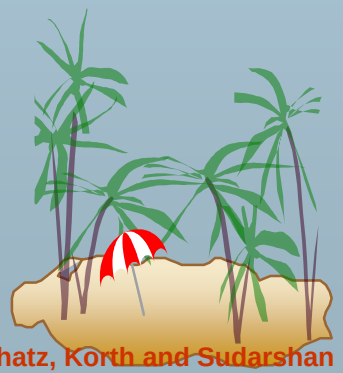


©Silberschatz, Korth and Sudarshan

B^+-Tree Index Files (Cont.)

A B^+-tree is a rooted tree satisfying the following properties:

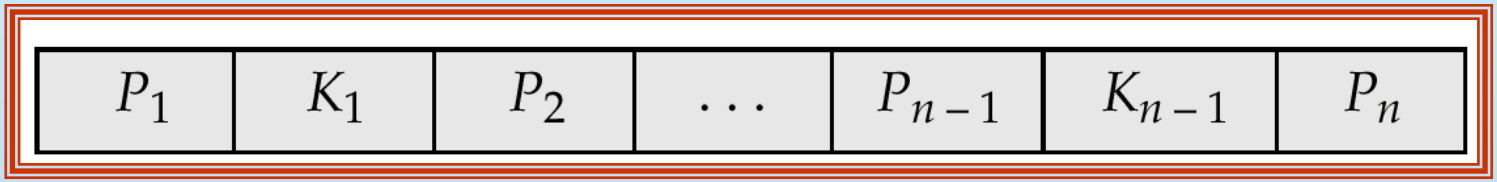
- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $[n/2]$ and n children.
- A leaf node has between $[(n–1)/2]$ and $n–1$ values
- Special cases:
  - If the root is not a leaf, it has at least 2 children.
  - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n–1)$ values.



©Silberschatz, Korth and Sudarshan

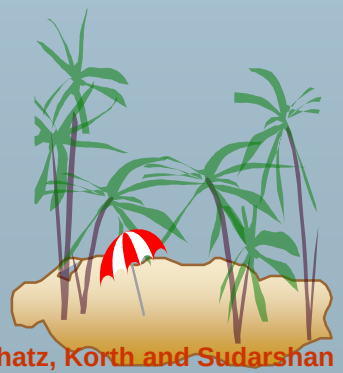
B+-Tree Node Structure

- Typical node



  - K_i are the search-key values
  - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \ldots < K_{n-1}$$

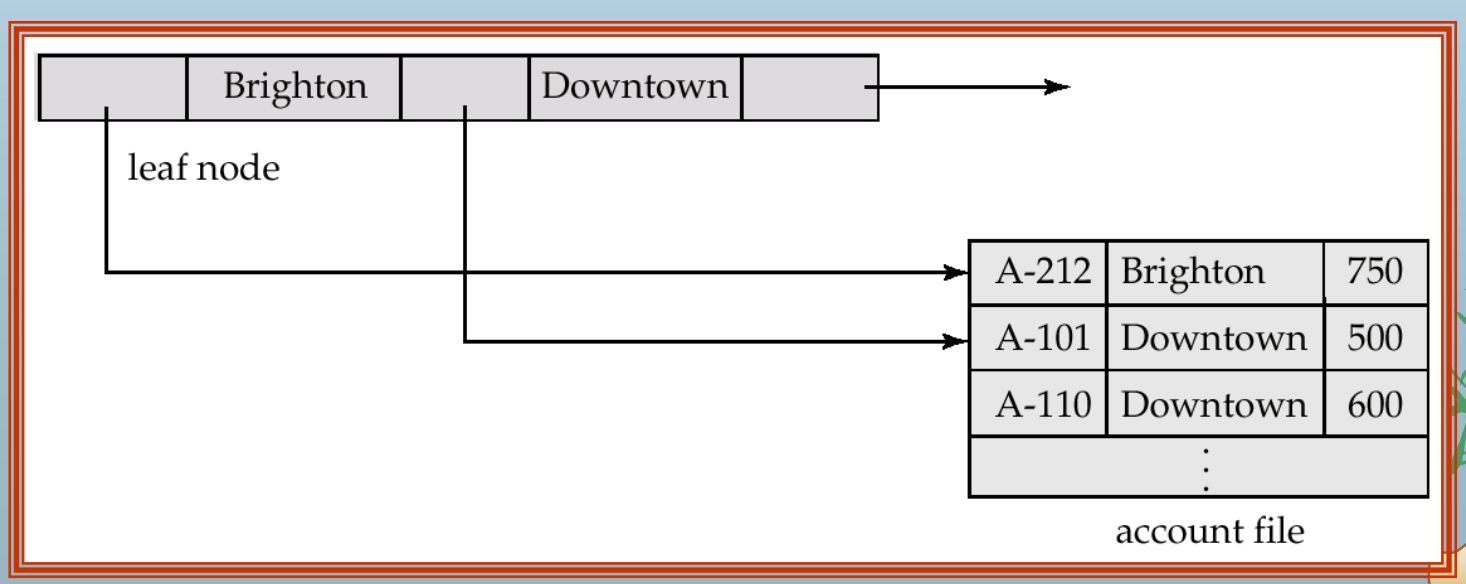


©Silberschatz, Korth and Sudarshan

Leaf Nodes in B+-Trees

Properties of a leaf node:

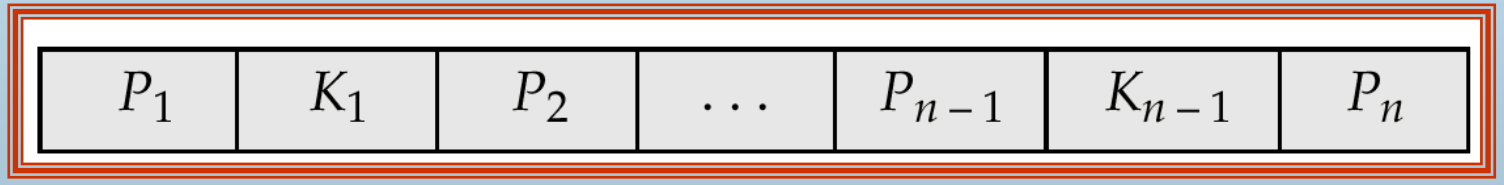
- For $i = 1, 2, \ldots, n-1$, pointer P_i points to a file record with search-key value K_i.
- If L_i, L_j are leaf nodes and $i < j$, L_i's search-key values are less than L_j's search-key values
- P_n points to next leaf node in search-key order



©Silberschatz, Korth and Sudarshan

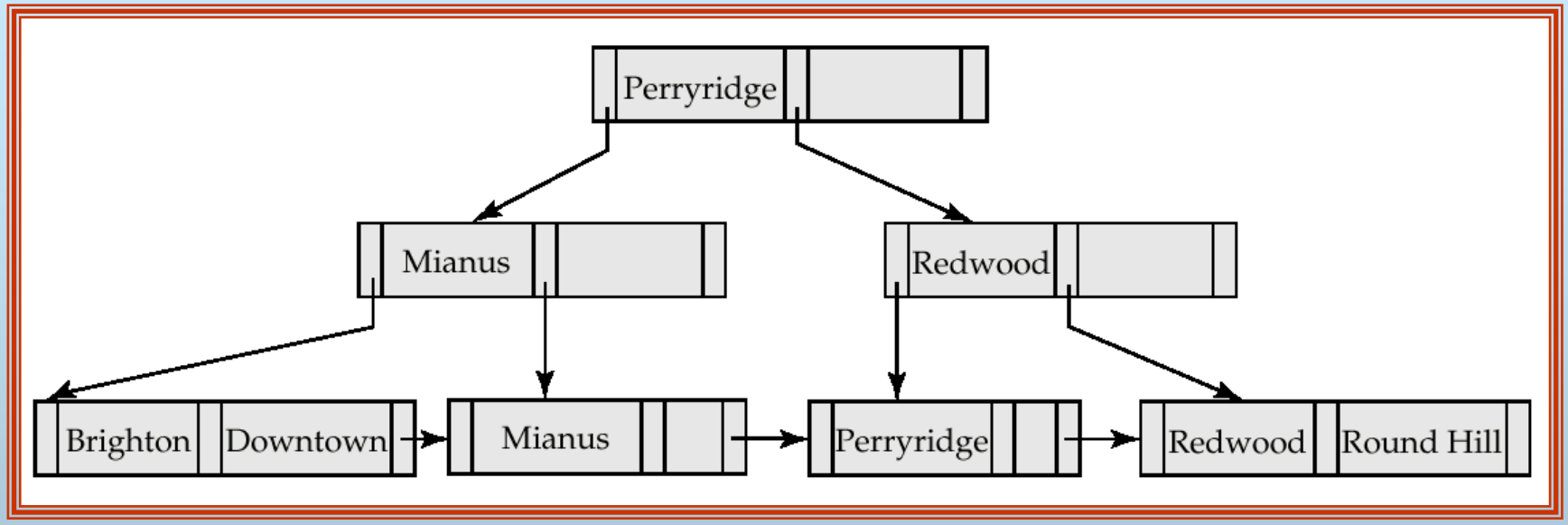
Non-Leaf Nodes in B+-Trees

- Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with *m* pointers:
  - All the search-keys in the subtree to which P_1 points are less than K_1
  - For $2 \leq i \leq n - 1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_{m-1}



©Silberschatz, Korth and Sudarshan

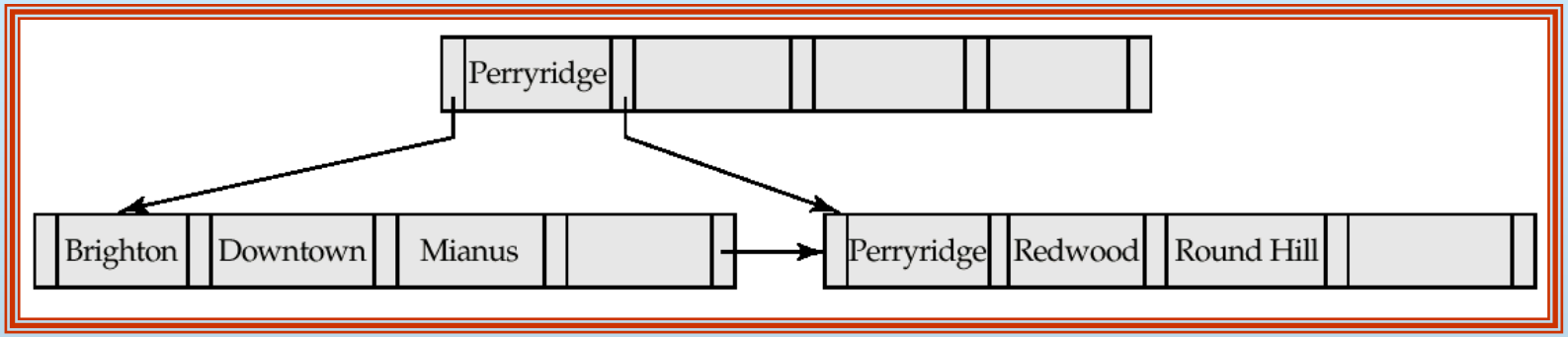
Example of a B+-tree



B+-tree for *account* file (*n* = 3)

©Silberschatz, Korth and Sudarshan

Example of B+-tree



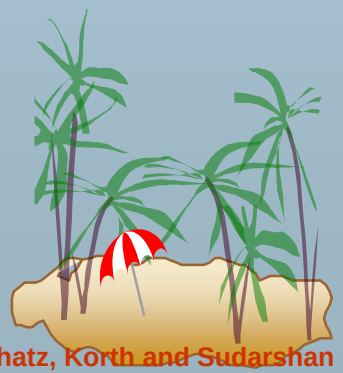
B+-tree for *account* file (*n* - 5)

- Leaf nodes must have between 2 and 4 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 5$).
- Non-leaf nodes other than root must have between 3 and 5 children ($\lceil n/2 \rceil$ and n with $n = 5$).
- Root must have at least 2 children.

©Silberschatz, Korth and Sudarshan

Observations about B^+-trees

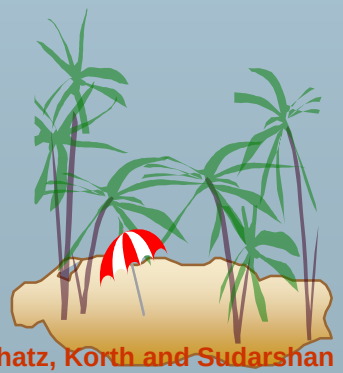
- Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.
- The non-leaf levels of the B^+-tree form a hierarchy of sparse indices.
- The B^+-tree contains a relatively small number of levels (logarithmic in the size of the main file), thus searches can be conducted efficiently.
- Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time (as we shall see).



©Silberschatz, Korth and Sudarshan

Queries on B^+-Trees

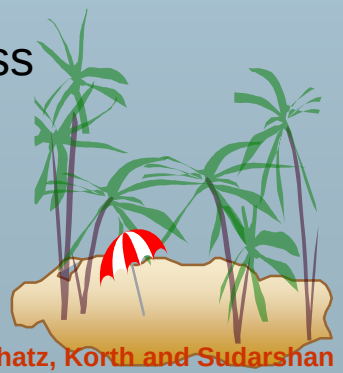
- Find all records with a search-key value of *k.*
  1. Start with the root node
     1. Examine the node for the smallest search-key value > *k.*
     2. If such a value exists, assume it is K_j. Then follow P_i to the child node
     3. Otherwise $k \geq K_{m-1}$, where there are *m* pointers in the node. Then follow P_m to the child node.
  2. If the node reached by following the pointer above is not a leaf node, repeat the above procedure on the node, and follow the corresponding pointer.
  3. Eventually reach a leaf node. If for some *i*, key K_i = k follow pointer P_i to the desired record or bucket. Else no record with search-key value *k* exists.



©Silberschatz, Korth and Sudarshan

Queries on B+-Trees (Cont.)

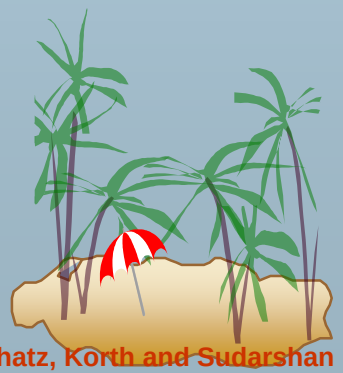
- In processing a query, a path is traversed in the tree from the root to some leaf node.
- If there are K search-key values in the file, the path is no longer than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.
- A node is generally the same size as a disk block, typically 4 kilobytes, and n is typically around 100 (40 bytes per index entry).
- With 1 million search key values and n = 100, at most $log_{50}(1{,}000{,}000) = 4$ nodes are accessed in a lookup.
- Contrast this with a balanced binary tree with 1 million search key values — around 20 nodes are accessed in a lookup
  - above difference is significant since every node access may need a disk I/O, costing around 20 milliseconds!



©Silberschatz, Korth and Sudarshan

Updates on B$^+$-Trees: Insertion

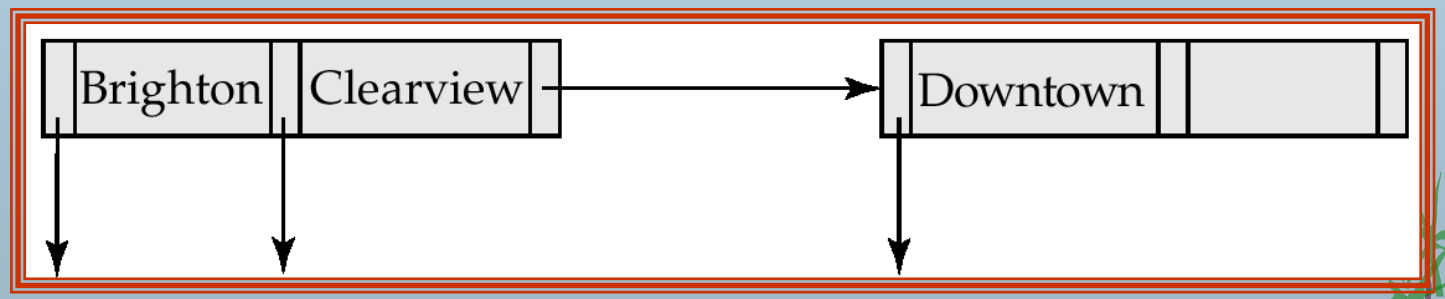
- Find the leaf node in which the search-key value would appear
- If the search-key value is already there in the leaf node, record is added to file and if necessary a pointer is inserted into the bucket.
- If the search-key value is not there, then add the record to the main file and create a bucket if necessary. Then:
  - If there is room in the leaf node, insert (key-value, pointer) pair in the leaf node
  - Otherwise, split the node (along with the new (key-value, pointer) entry) as discussed in the next slide.



©Silberschatz, Korth and Sudarshan

Updates on B+-Trees: Insertion (Cont.)

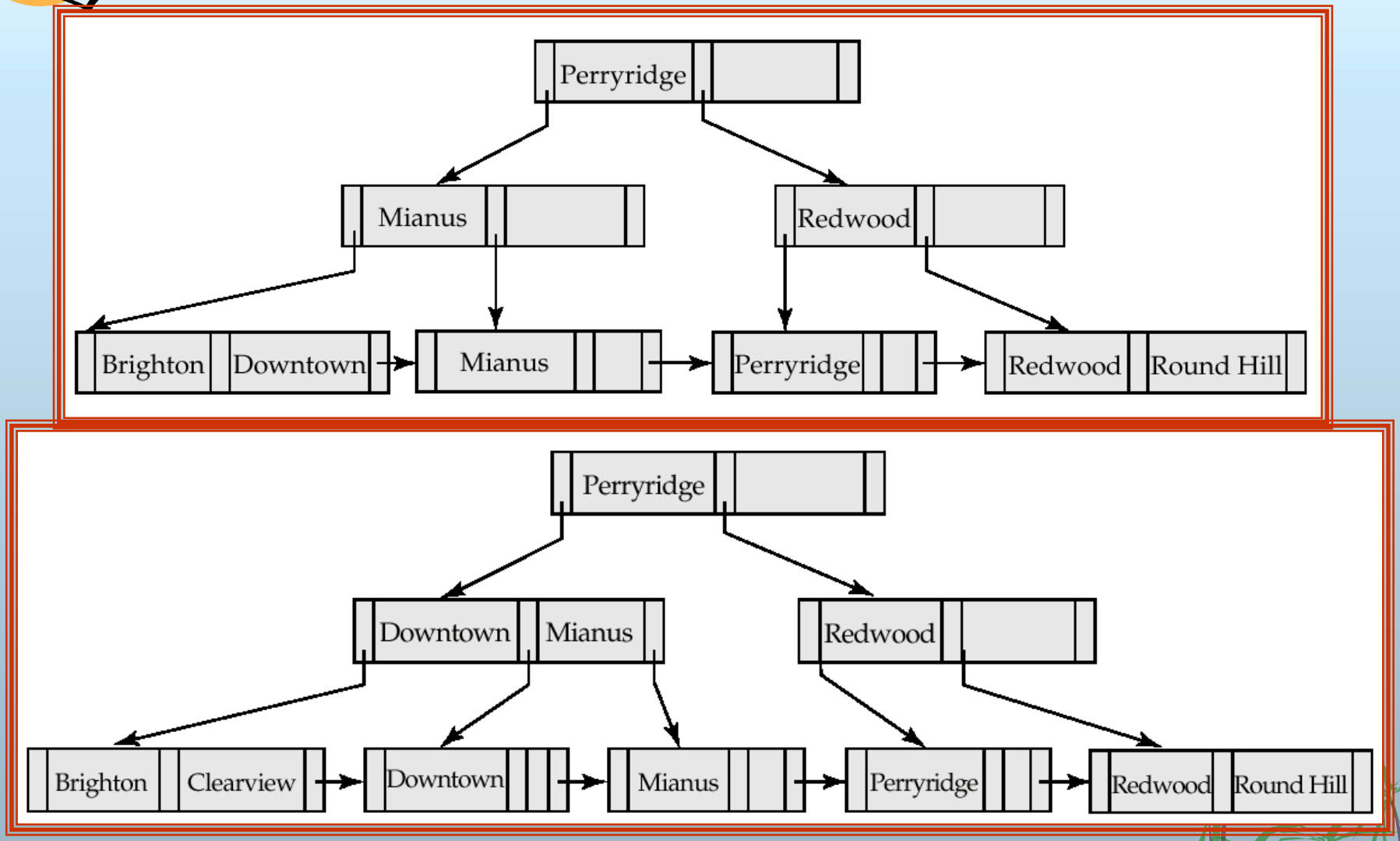
- Splitting a node:
  - take the *n*(search-key value, pointer) pairs (including the one being inserted) in sorted order. Place the first $\lceil n/2 \rceil$ in the original node, and the rest in a new node.
  - let the new node be *p,* and let *k* be the least key value in *p.* Insert (*k*,*p*) in the parent of the node being split. If the parent is full, split it and propagate the split further up.
- The splitting of nodes proceeds upwards till a node that is not full is found. In the worst case the root node may be split increasing the height of the tree by 1.



Result of splitting node containing Brighton and Downtown on inserting Clearview

©Silberschatz, Korth and Sudarshan

Updates on B+-Trees: Insertion (Cont.)

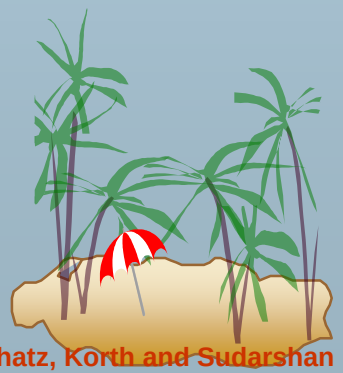


B+-Tree before and after insertion of "Clearview"

©Silberschatz, Korth and Sudarshan

Updates on B$^+$-Trees: Deletion

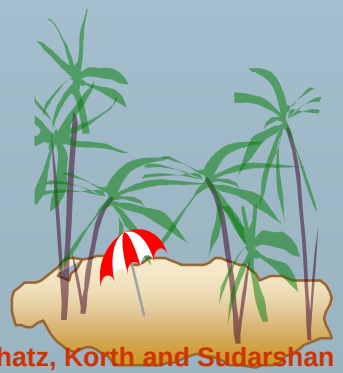
- Find the record to be deleted, and remove it from the main file and from the bucket (if present)
- Remove (search-key value, pointer) from the leaf node if there is no bucket or if the bucket has become empty
- If the node has too few entries due to the removal, and the entries in the node and a sibling fit into a single node, then
  - Insert all the search-key values in the two nodes into a single node (the one on the left), and delete the other node.
  - Delete the pair (K_{i-1}, P_i), where P_i is the pointer to the deleted node, from its parent, recursively using the above procedure.



©Silberschatz, Korth and Sudarshan

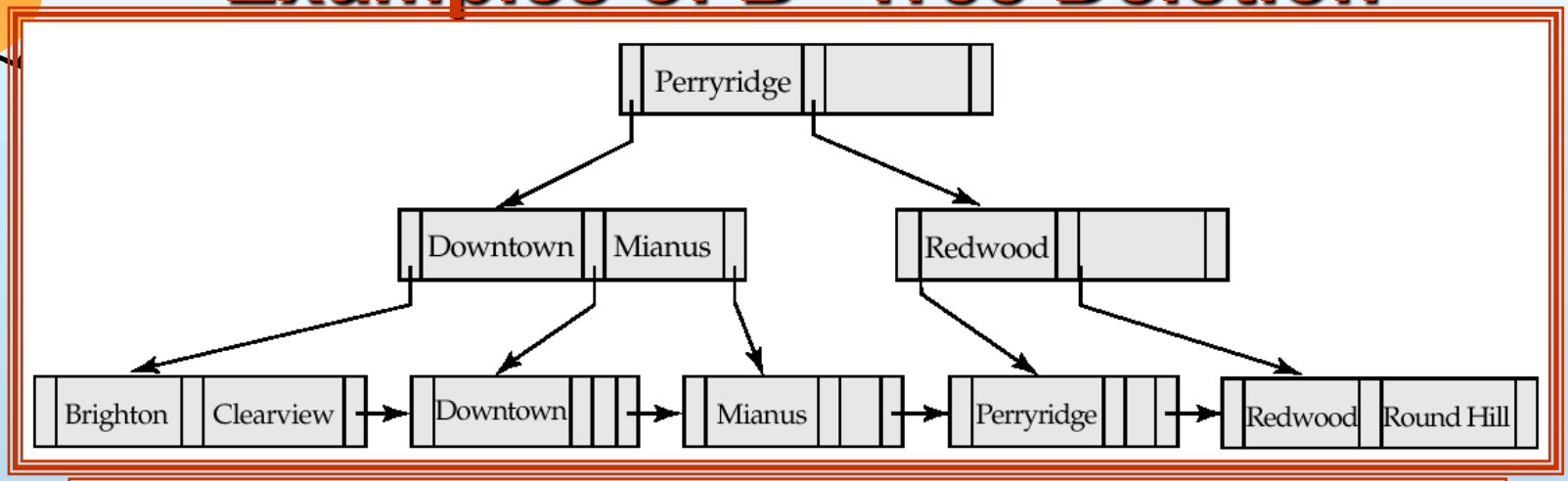
Updates on B+-Trees: Deletion

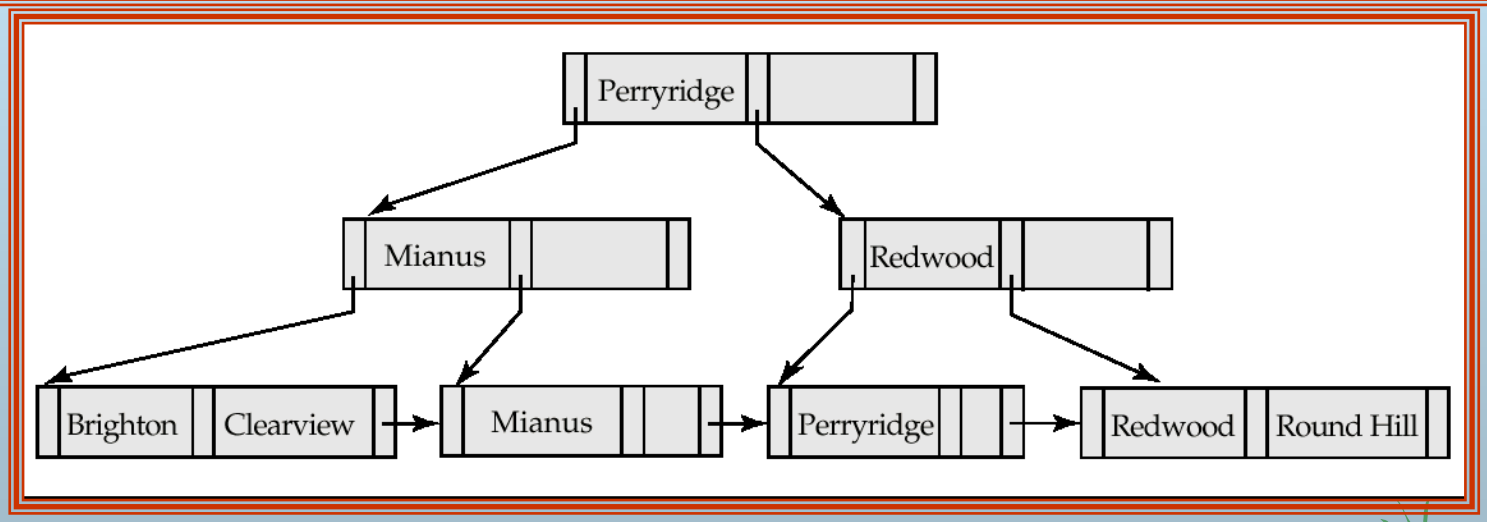
- Otherwise, if the node has too few entries due to the removal, and the entries in the node and a sibling do not fit into a single node, then
  - Redistribute the pointers between the node and a sibling such that both have more than the minimum number of entries.
  - Update the corresponding search-key value in the parent of the node.
- The node deletions may cascade upwards till a node which has $\lceil n/2 \rceil$ or more pointers is found. If the root node has only one pointer after deletion, it is deleted and the sole child becomes the root.



©Silberschatz, Korth and Sudarshan

Examples of B+-Tree Deletion



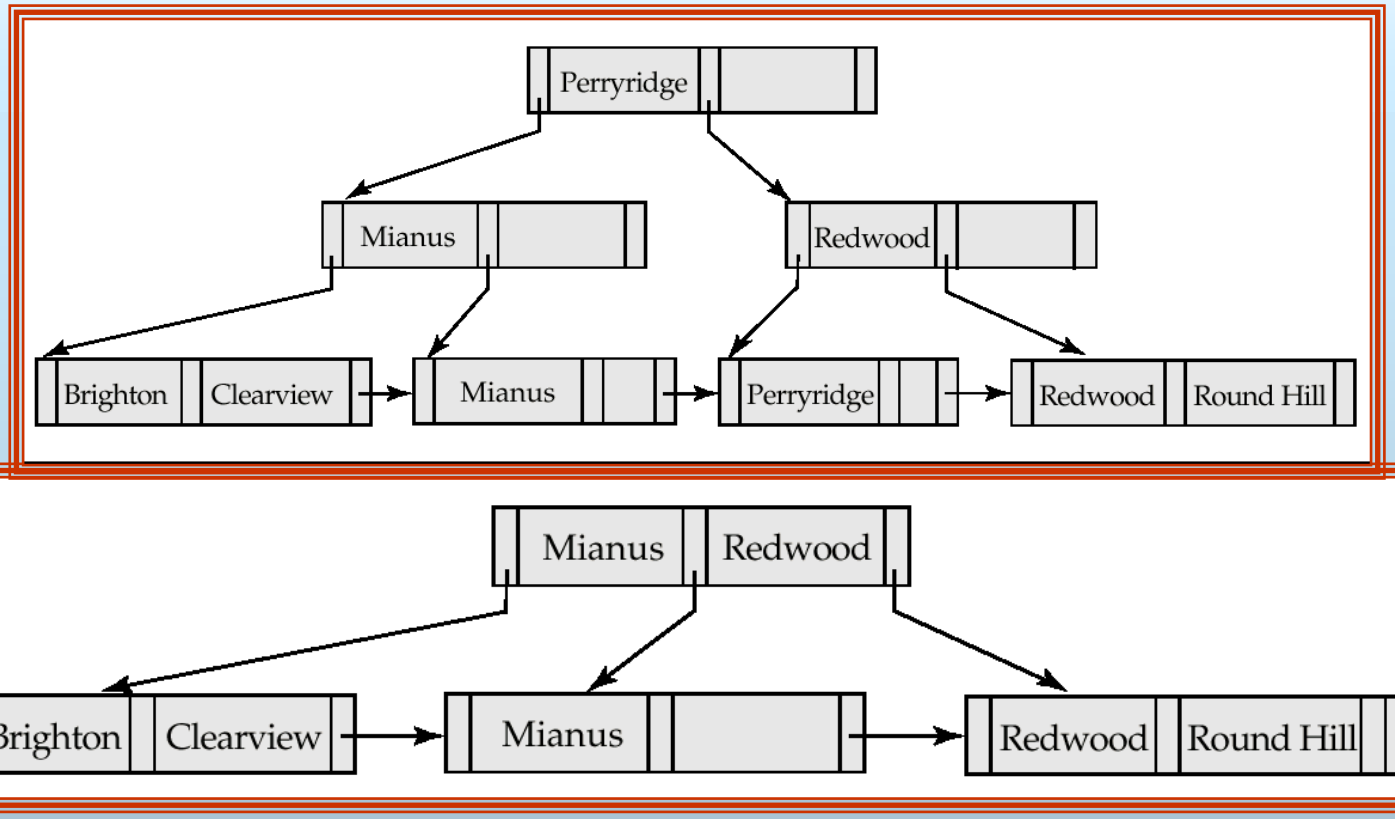


Before and after deleting “Downtown”

- The removal of the leaf node containing “Downtown” did not result in its parent having too little pointers. So the cascaded deletions stopped with the deleted leaf node’s parent.

©Silberschatz, Korth and Sudarshan

Examples of B+-Tree Deletion (Cont.)

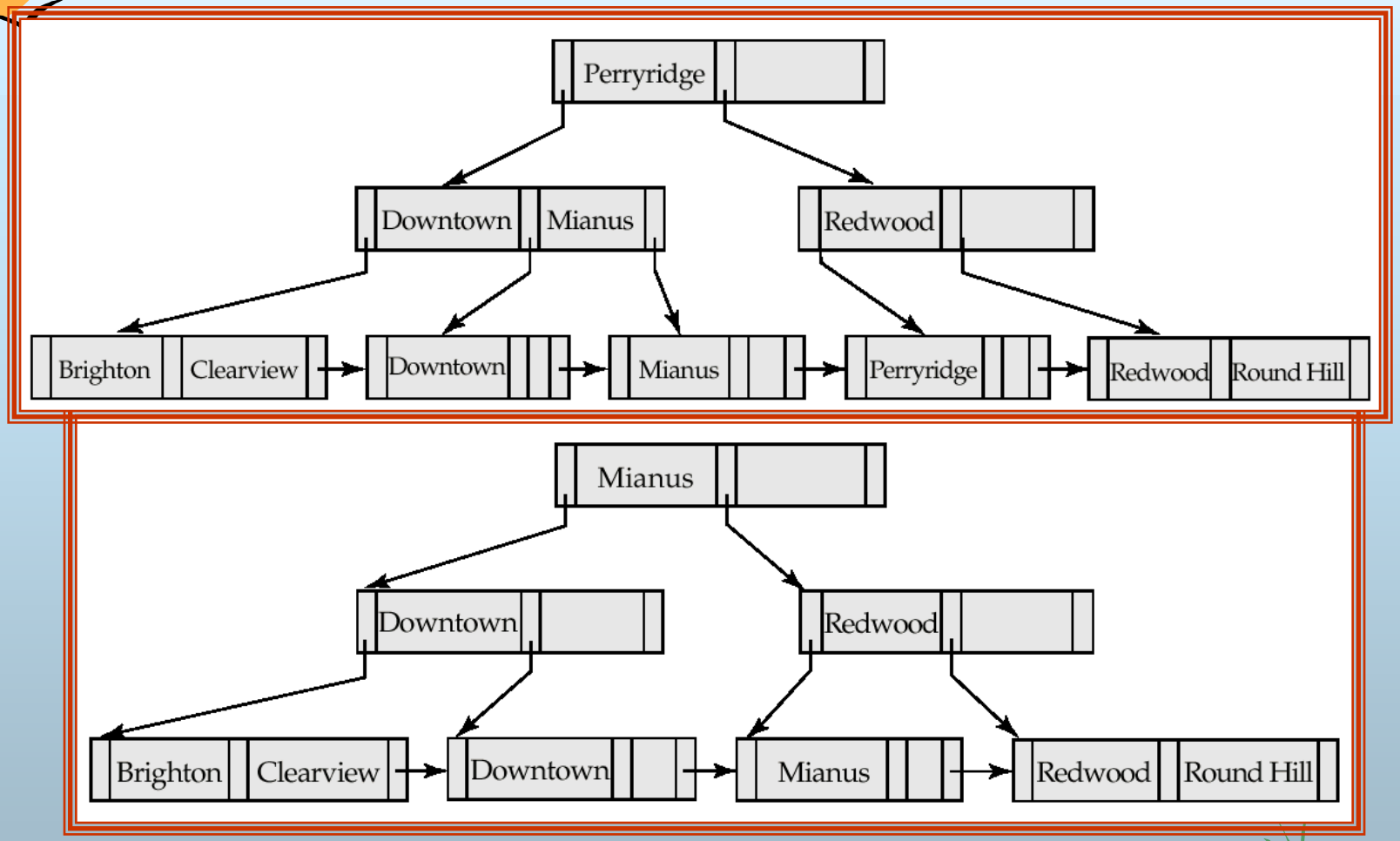


Deletion of "Perryridge" from result of previous example

- Node with "Perryridge" becomes underfull (actually empty, in this special case) and merged with its sibling.
- As a result "Perryridge" node's parent became underfull, and was merged with its sibling (and an entry was deleted from their parent)
- Root node then had only one child, and was deleted and its child became the new root node

©Silberschatz, Korth and Sudarshan

Example of B+-tree Deletion (Cont.)



Before and after deletion of "Perryridge" from earlier example

- Parent of leaf containing Perryridge became underfull, and borrowed a pointer from its left sibling
- Search-key value in the parent's parent changes as a result

©Silberschatz, Korth and Sudarshan